

# Exchange Health Check — Guida utente

Monitoraggio periodico per ambienti Exchange on-premises multi-server, con alerting via e-mail. Controlla servizi, spazio disco, stato del DAG e delle copie di database, code di trasporto, certificati e Managed Availability. Lavora in **sola lettura**: nessun cmdlet di scrittura verso Exchange.

Repository: <https://github.com/diepo/ExchangeHealthCheck>

## 1. Cosa controlla

| Area                 | Dettaglio                                                          |
|----------------------|--------------------------------------------------------------------|
| Connettività         | Server raggiungibile via WinRM/CIM                                 |
| Sistema operativo    | Uptime, reboot in sospeso, memoria, CPU                            |
| Disco                | Spazio libero su ogni volume, incluse le mount point senza lettera |
| Servizi              | Servizi Exchange fermi che dovrebbero essere in esecuzione         |
| Componenti           | Server lasciato in maintenance mode dopo un patching               |
| Managed Availability | Health set non sani, con i monitor coinvolti in dettaglio          |
| DAG                  | Membri fermi, witness, nodi del cluster                            |
| Copie di database    | Stato delle copie, copy/replay queue, content index                |
| Replica              | <code>Test-ReplicationHealth</code>                                |
| Database             | Database non montati, età dell'ultimo backup                       |
| Code di trasporto    | Code accumulate, submission, poison, retry, back pressure          |
| Certificati          | Scadenza, stato non valido, duplicati che confondono i connector   |

## 2. Requisiti

- Windows PowerShell 5.1 o superiore (va bene anche PowerShell 7.x)
- Da eseguire preferibilmente **su un server Exchange**, oppure da una macchina con `Organization.ConnectTo` configurato per aprire una remote session
- Account con ruolo RBAC **View-Only Organization Management** e amministratore locale sui server (serve per il fan-out WinRM)
- WinRM abilitato sui server target

### 3. Installazione

---

1. Copia la cartella del progetto su un server di gestione o su un server Exchange.
2. Crea la tua configurazione a partire dal template:

```
powershell Copy-Item .\ExchangeHealthCheck.config.example.json  
.\ExchangeHealthCheck.config.json
```

Il file `ExchangeHealthCheck.config.json` contiene dati del tuo ambiente (nomi server, indirizzi mail) e non va mai condiviso o versionato.

3. Apri il file e imposta almeno le chiavi seguenti:
- `Organization.ViewEntireForest`: `false` se la tua organizzazione Exchange vive in un solo dominio (caso più comune)
  - `Servers.Include`: i server da controllare (o `["*"]` per tutti)
  - `Mail.SmtptServers`, `Mail.From`, `Mail.To`

### 4. Primo test, passo per passo

---

#### 4.1 — Un solo check, senza inviare nulla

```
.\Invoke-ExchangeHealthCheck.ps1 -Server NOME-SERVER -Check Disk -NoMail -Verbose
```

Se questo funziona, sono confermati i tre presupposti di base: i cmdlet Exchange rispondono, WinRM verso quel server funziona, i dati tornano.

#### 4.2 — Tutti i check, ancora senza inviare nulla

```
.\Invoke-ExchangeHealthCheck.ps1 -NoMail -Verbose
```

Con `-NoMail` lo stato degli alert non viene mai marcato come notificato: puoi ripetere il test tutte le volte che vuoi senza perdere alert veri quando attiverai davvero la mail.

#### 4.3 — Leggi il report

Ogni esecuzione produce un CSV in `Reports\`:

```
$ultimo = Get-ChildItem .\Reports\*.csv |  
    Sort-Object LastWriteTime -Descending |  
    Select-Object -First 1  
  
Import-Csv $ultimo.FullName |  
    Where-Object Severity -ne 'OK' |  
    Format-Table Severity, Server, Category, Message -AutoSize
```

Usa questo passaggio per tarare le soglie e popolare `Ignore.*` prima di attivare le mail: il primo giro con la mail attiva invia tutte le anomalie già presenti in quel momento, in un unico messaggio.

#### 4.4 — Prova solo l'invio mail

```
.\Invoke-ExchangeHealthCheck.ps1 -TestMail
```

Non esegue alcun controllo: manda solo un messaggio di prova e mostra i parametri SMTP effettivamente usati. Se fallisce, il log riporta la causa reale (host non risolvibile, connessione rifiutata, destinatario non accettato, autenticazione richiesta).

#### 4.5 — Primo giro vero

```
.\Invoke-ExchangeHealthCheck.ps1
```

## 5. Come funziona l>alerting

Lo stato di ogni anomalia è persistito in `State\alert-state.json` con chiave `Categoria|Server|Oggetto`. Ad ogni esecuzione:

- **anomalia nuova** → mail immediata
- **anomalia peggiorata** (es. `Warning` → `Critical`) → mail immediata
- **anomalia già nota, non peggiorata** → nessuna mail finché non scade `Alerting.CooldownMinutes` (default 120 minuti), poi un promemoria
- **anomalia rientrata** → mail di rientro con la durata del disservizio
- **nessuna anomalia da giorni** → un heartbeat ogni `Alerting.HeartbeatHours` (default 24), per sapere che il monitor è vivo

Se l'invio della mail fallisce, l'anomalia **non** viene marcata come notificata: il giro successivo riprova, non resta in silenzio per il cooldown.

**Colpo d'occhio sull'ambiente.** La mail (e la console, durante un giro a secco) apre con un riepilogo **per categoria** (`Disk`, `Service`, `Queue`, `Certificate`, ...): quante cose sono `Critical`/`Warning` in ciascuna, prima ancora del dettaglio server per server. Risponde a "cosa non va nell'infrastruttura" senza dover aprire ogni tabella.

**Due mail distinte.** Oltre alla mail di riepilogo (sempre completa: contatori, categorie, server, dettaglio), quando in un giro compare qualcosa di davvero nuovo o peggiorato ne parte una seconda, minimale, con oggetto **"WARNING FOUND"** e solo le righe rilevanti — pensata per essere letta in pochi secondi. Scatta solo su:

- un **nuovo Critical** in qualunque categoria (o un `Warning` che diventa `Critical`);
- una **coda** che raggiunge `Thresholds.QueueSubjectThreshold` (default 200 messaggi), a prescindere dalla sua severità.

Non scatta mai su un promemoria di un'anomalia già nota: una coda che oscilla di poco (es. 15→18 messaggi) non genera questa mail.

**Oggetto con tag dedicato per le code.** Se una qualunque coda raggiunge quella stessa soglia di 200 messaggi, l'oggetto della mail di riepilogo guadagna il tag **"ATTENZIONE CODE"**, visibile senza aprire il messaggio.

## 6. Programmazione automatica

```
.\Install-ExchangeHealthCheckTask.ps1 `
  -IntervalMinutes 15 -UserName 'DOMINIO\account-servizio'
```

Registra un'attività pianificata che parte all'avvio del server e si ripete ogni N minuti. Un mutex globale impedisce esecuzioni sovrapposte se un giro dura più dell'intervallo.

Per rimuoverla: `.\Install-ExchangeHealthCheckTask.ps1 -Unregister`

## 7. Domande frequenti

**"WinRM cannot find the computer" su un server acceso e funzionante** Quasi sempre è risoluzione nome, non un server giù. Verifica con l'FQDN: `Invoke-Command -ComputerName server.tuodominio.local { $env:COMPUTERNAME }`. Lo script usa già l'FQDN di default (`Servers.UseFqdnForRemoting`).

**Lo script gira su un server e proprio quel server risulta irraggiungibile** Comportamento noto e già gestito: il server locale non passa da WinRM, viene interrogato direttamente. Se accade, aggiorna alla versione più recente.

**"Nessun DAG rilevato"** Normale su un server standalone senza DAG: i check DAG/replica vengono saltati, il resto continua.

**"object ... could not be found on "** Prova `"Organization": { "ViewEntireForest": false }`. Se persiste, imposta `PreferredDomainController` con l'FQDN di un DC della stessa foresta dei server Exchange (non basta la stessa foresta: serve anche che il DC replichi il Configuration NC). Nota anche che alcuni cmdlet (es. lo stato delle copie database) rispondono lo stesso messaggio quando il server semplicemente non ospita copie, o quando l'oggetto è fuori dallo scope RBAC: non è sempre un problema di DC.

**Back pressure segnalata come "Low"** Low è lo stato sano. Se lo vedi ancora segnalato come anomalia, aggiorna lo script: era un bug risolto (i campi venivano letti invertiti).

**ForwardSyncDaemon o ProvisioningRps sempre Inactive** Normale: sono componenti usati solo dal datacenter Microsoft, su on-premises restano Inactive per progetto. Sono esclusi di default.

**Due certificati con lo stesso nome, uno scaduto** I connector Exchange identificano i certificati per Subject+Issuer, non per thumbprint: con due certificati identici in quei campi, Exchange può agganciare quello sbagliato. Lo script segnala automaticamente i duplicati con questa caratteristica.

**"Mailbox unavailable / Unable to relay / Recipient not in accepted domain"** Il destinatario della mail non è in un dominio che Exchange considera suo: è relay, negato per l'invio anonimo. Usa una casella interna, oppure autentica l'invio (`Mail.UseDefaultCredentials` o `Mail.CredentialFile`), oppure crea un receive connector dedicato limitato per IP.

## 8. Sicurezza e privacy dei dati

- Lo script non modifica nulla in Exchange: solo cmdlet `Get-*` e `Test-*`.
- La configurazione reale, i log, i report CSV e lo stato degli alert contengono nomi di server, database e indirizzi del tuo ambiente: non vanno mai condivisi o versionati (già esclusi di default in `.gitignore`).
- Le credenziali SMTP, se usate, sono salvate con `Export-Clixml` (cifatura DPAPI legata all'account che le genera): vanno create con lo stesso account che eseguirà lo scheduled task.

---

*Documento generato insieme allo sviluppo dello script. Va aggiornato ad ogni modifica funzionale rilevante — vedi `HANDOFF.md` nel repository per il dettaglio tecnico completo e la cronologia dei fix.*